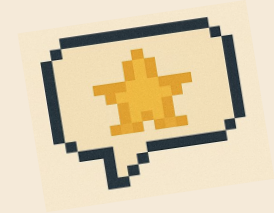
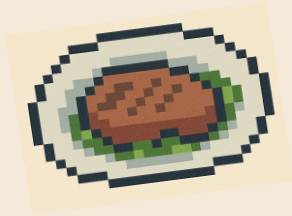


ReviewChew

A Restaurant Review Site



APAN 5400 Managing Data: Final Presentation

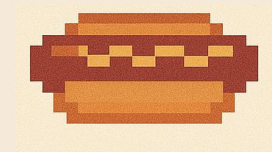
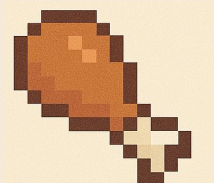
Arden Haggin (aph2163)

Garima Gupta (gg2971)

Kibaek Kim (kk3789)

Wendy Luu (bl3085)

Yi Ching (Sunny) Wang (yw4433)



Agenda



1 Background & Business Problem

2 Data Source & Procurement Process

3 Technologies

4 System Design

5 Success Metrics & Data Quality

6 Scalability & Projected Cost

7 Conclusion & Future Recommendations

8 Web App Demo



Background & Business Problem

Restaurant-goers are often overwhelmed by the sheer volume of information on review platforms such as Yelp or Tripadvisor, leading to indecision regarding their desired dining experience.

Solution: ReviewChew

Our solution, ReviewChew, aims to address the difficulties of traditional review platforms by offering a user-friendly, search-based service that provides AI summaries of restaurant reviews. With ReviewChew, customers can streamline their restaurant-picking process while still ensuring they have all the information needed to make an informed dining decision.



Data Details & Procurement

The data for our project has been procured from the Yelp Open Dataset, which is free via [Yelp Data Licensing](#) and intended for educational use.

With over 6 million reviews of over 100,000 businesses, the Yelp Open Dataset provides us with ample information to build a highly replicable, scalable solution.

Files	Details	Size
Businesses	Businesses information and attributes	100 MB
Reviews	Rating and review comments	4.74 GB
Users	ID, name, review count, credibility	2.72 GB
Photos	Business ID, photo, and caption	7.11 GB

Technologies



Flask

- Routes:
 - Result page: handles search queries and shows listings
 - Detail page: shows restaurants info, summary, & images.
- Fetches data from PostgreSQL (restaurant info i.e. address, ratings, cuisine, reviews,...), MongoDB (LLM summaries), Amazon S3 (images)

PostgreSQL

- Stores cleaned text data from Yelp, including reviews, and used as the input for info summarization.
- Queried by Flask for restaurant info data.

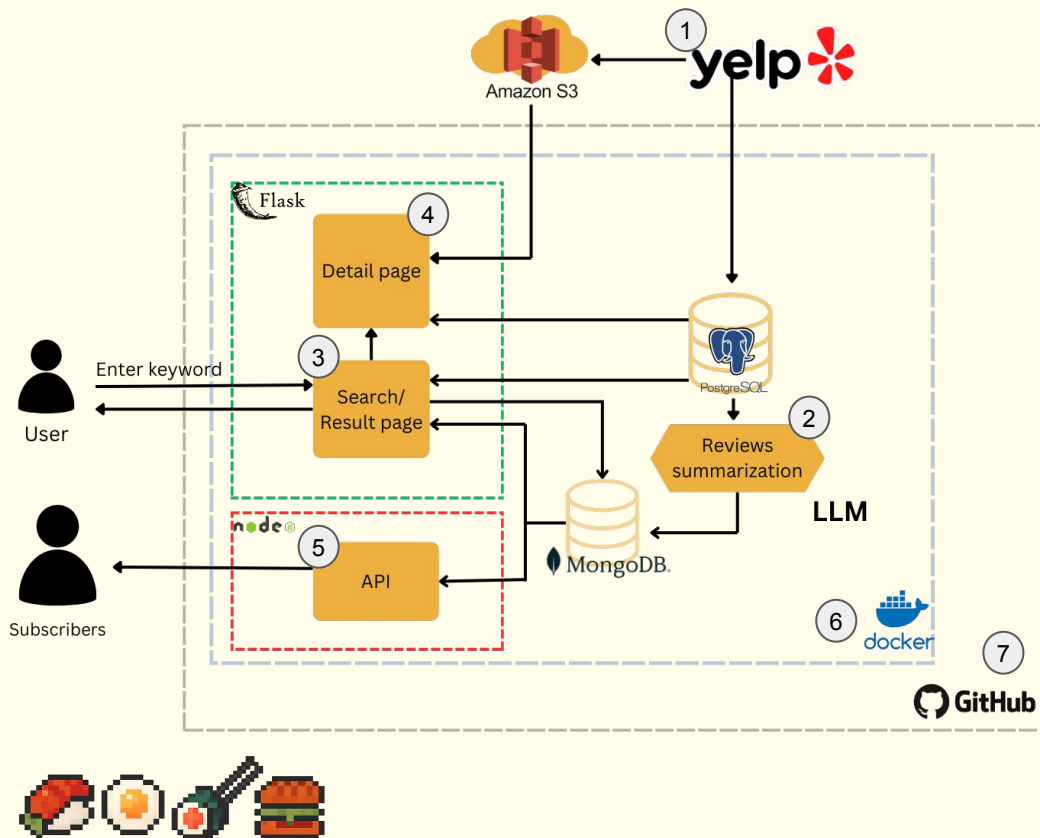
MongoDB

- Stores summarized data from LLM (i.e., ChatGPT)
- Queried by Flask to serve quick JSON-like results

Docker

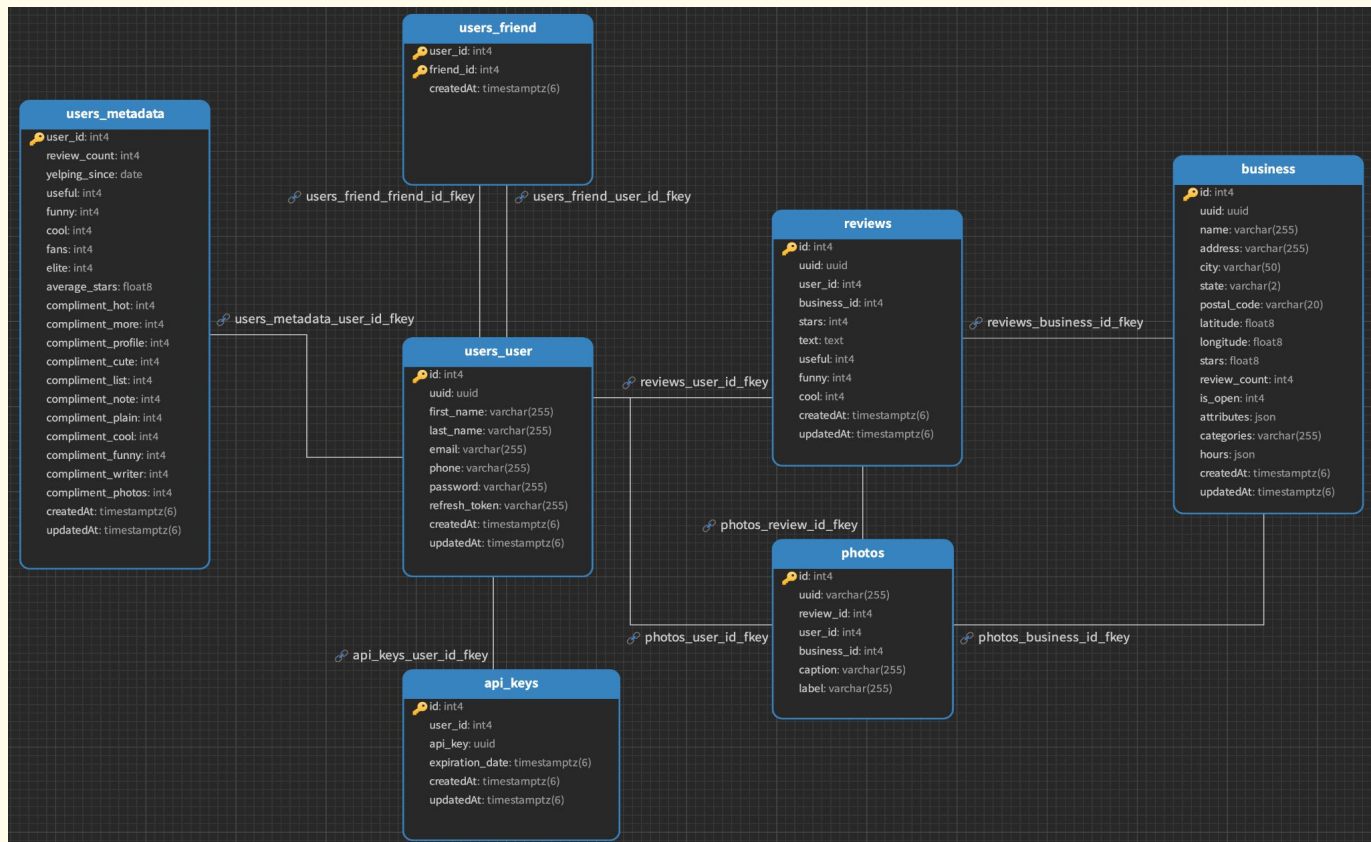
- Containers wrap services to isolate environments
- Used for: PostgreSQL, MongoDB, Flask App, LLM, NodeJS API

System Design



- ① Yelp's data is continuously streamed into PostgreSQL; images uploaded to Amazon S3.
- ② Reads new reviews from PostgreSQL. Sends review text to an LLM. Stores summaries in MongoDB as json documents.
- ③ Takes user search query. Queries MongoDB for summaries and PostgreSQL for metadata. Displays a list of matched businesses with summarized reviews.
- ④ Triggered when a user selects an item. Fetches full data from PostgreSQL, summary from MongoDB, and image links from Amazon S3. Renders detailed business/review info for the user.
- ⑤ Handles external programmatic access via REST API. Our paid subscribers have access to this API to access our restaurant summarized data.
- ⑥ Docker is used to ensure isolated, portable environments for PostgreSQL, MongoDB, Flask app, Node.js API, and Summarization pipeline.
- ⑦ All code for Flask frontend, API backend, Summarization job, Docker configs is managed via GitHub for collaboration and deployment.

System Design (PostgreSQL Model)





Success Metrics

	User Engagement	System Performance
Quantitative	- # of downloads - Daily active users, monthly active users (DAU/MAU) - Repeat users - Query frequency (monthly search volume) - Time on website, click-rate	- Query response time - Crash rate - API latency
Qualitative	- UI/UX user satisfaction - User feedback on the query result - Quality of restaurant summaries	- User's perception of speed - System stability



Data Quality

Dimensions	
Accuracy	Tagging errors (human mistakes or multi-purpose venues. Gas stations and convenience stores mislabeled as “restaurants”. Resulted in false positives, showing non-restaurants on the map
Completeness	Only used subset of Yelp Data due to restrictions on API calls (paid).
Consistency	Reviews summarization quality depends on LLM models.
Uniqueness	LLM outputs are unique; combination of LLM and given Yelp dataset.
Timeliness	Timely since data generated by LLM are saved in MongoDB (Json).



Scalability & Projected Cost

Component	Cloud Service	Role	Scalable With	Estimated Monthly Cost
PostgreSQL	AWS RDS	Stores reviews, metadata	Read/write load, data volume	2 core, 16GB RAM, 100GB SSD: \$274.85 / month
MongoDB	MongoDB Atlas	Stores LLM summaries	Document volume, query frequency	400-500 Ops/Sec: \$30 / Month
Amazon S3	AWS S3	Hosts restaurant images	Number of listings, image traffic	\$1-30+ depending on usage Free (Preferred Method)
Application Server	AWS EC2	Hosts Flask frontend and APIs	User traffic	\$25+/month and auto scalable
LLM Summarization	Local/OpenAI API	Summarizes reviews via model pipeline	Review volume, token usage	1M tokens : \$1 Current level: Free*

We prioritized future scalability by using cloud-native services that allow each system component to scale independently based on demand for user queries, review summarization, and restaurant data retrieval. Cloud computing resources such as AWS RDS, EC2, S3, and MongoDB Atlas support increasing traffic, data volume, and processing requirements efficiently.

Total cost starts from **\$332+ monthly** depending on traffic loads, storage volume, zone pricing, and team size. Revenue from ads or sponsorships not considered, but would likely offset costs entirely.



Final Takeaways

Conclusion	Recommendations
• The application successfully aggregates and maps review data but remains narrowly focused on restaurants. • Natural-language processing is constrained by current CPU resources, limiting LLM capabilities. • The Flask-based front end functions but needs UX/UI enhancements and deeper map integration. • Data security and governance measures are not yet fully implemented.	• Platform expansion: Extend support to other review-based services (e.g., hotels, salons). • Data integration: Pull location and review data directly from third-party APIs. • LLM upgrade: Adopt a more powerful language model or leverage cloud-based resources to overcome CPU limits. • UI/UX overhaul: Rebuild the Flask interface with Bootstrap; embed interactive maps on detail pages. • Security & governance: Encrypt the database and establish comprehensive safety protocols. • Session management: Implement token-based authentication to secure user sessions and API access. • Bookmarking feature: Allow users to save and manage favorite locations directly on the map.

Web App Demo



 **ReviewChew**

Home Map Services Contact 

Let ReviewChew find a restaurant for you

ReviewChew
Craving the best? Let AI serve you the highlights from real reviews.

Quick Links
[Home](#)
[About](#)
[Services](#)
[Contact](#)

Find Us on LinkedIn
[Kibwek Kim](#)
[Aiden Haggin](#)
[Wendy Liu](#)
[Sunny Wang](#)
[Garima Gupta](#)

POST /business/getBusinessListInCity Business List

Get Business List in specific city

Parameters

Name	Description
body required	State and City
object	Edit Value Model
(body)	<pre>{ "state": "NJ", "city": "Beverly" }</pre>

Parameter content type

Responses

Code	Description
200	Result of Business List in the City
Example Value	Model
	<pre>{ "success": true, </pre>

Web Application

<http://hyper07.ddns.net:5010/>

API Docs

<http://hyper07.ddns.net:3100/api/v1/api-docs/#/>



Team Responsibilities

Member	Responsibilities
Arden	Flask Web Application: 1) Backend: Set up flask routes for queries, results, etc. (app.py), wrote functions to load and process data, handled requests and responses, 2) Frontend: HTML templates for home, map, services, and contact.
Kibaek	1) Architecting database, 2) Building backend application, and 3) Integrating multiple tech stacks in a docker
Garima	Flask Web Application: 1) Backend: Fetching data via API calls and developed an interactive Leaflet.js map interface with state and city dropdowns, plotting each restaurant 2) Frontend: HTML template for map
Wendy	Flask Web Application: 2) Backend: LLM summary of restaurant reviews from Postgres using Open Llama, 2) Frontend: HTML template for restaurant details, results
Sunny	Flask Web Application: 1) Frontend: HTML template for login, management of static files and creation of UX elements